

Concurrency

Motivation

- Operating systems need to be able to handle *Multiple Things At Once* (MTAO)
 - processes, interrupts, background system maintenance
- Servers need to handle MTAO
 - Multiple connections handled simultaneously
- Parallel programs need to handle MTAO
 - To achieve better performance
- Programs with user interfaces often need to handle MTAO
 - To achieve user responsiveness while doing computation
- Network and disk bound programs need to handle MTAO
 - To hide network/disk latency

Motivations

A programmer usually needs to write programs that manage MTAO.

For example, consider a word processor:

- Reads input from keyboard
- Writes on screen
- Formats the data structure
- Periodically saves on disk
- Spell checking
-

Motivations

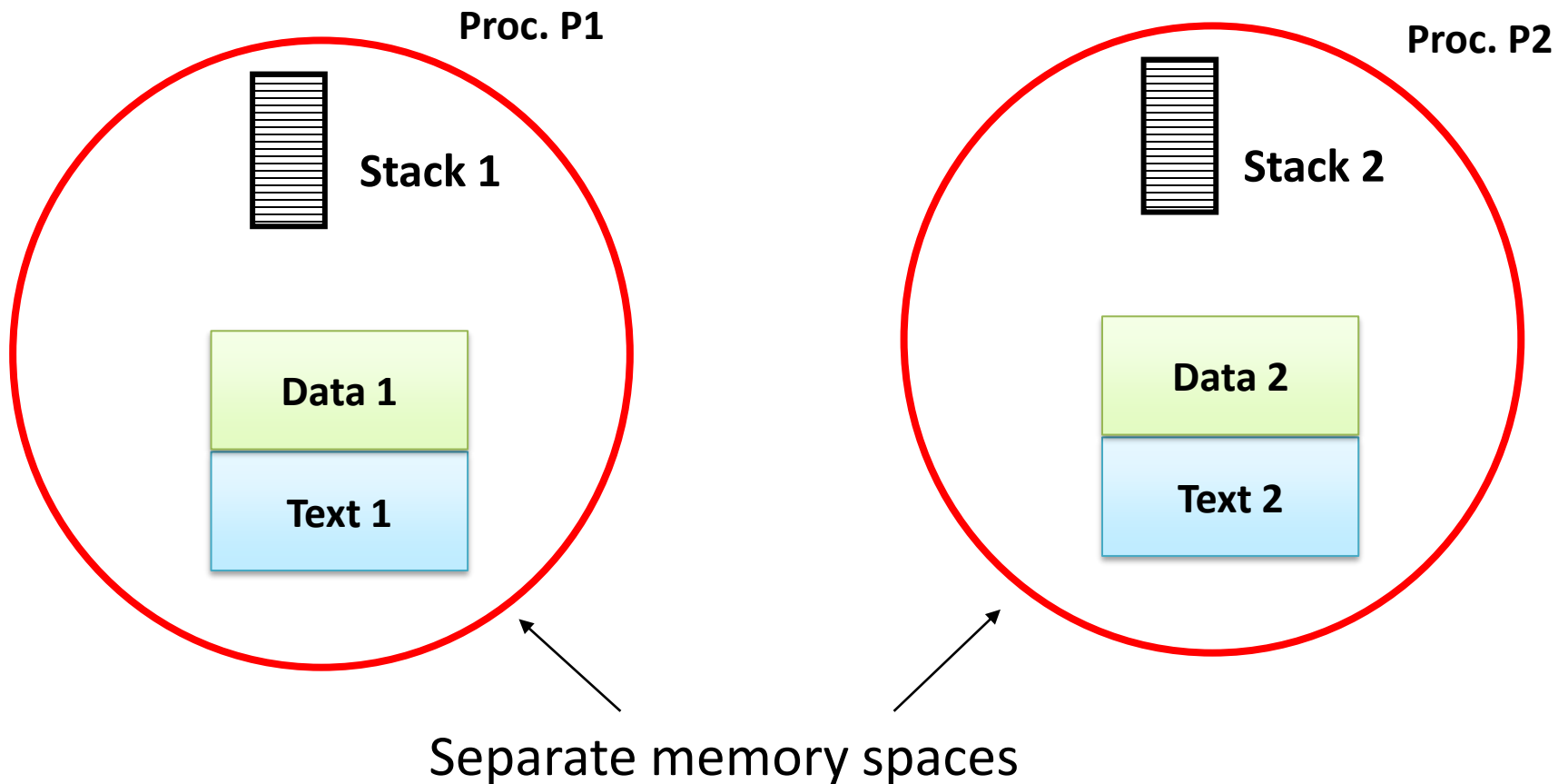
Idea: implement each function in a separate process

However:

- Each process has its own private memory space
- Processes can cooperate by using inter-process communication (IPC) mechanisms offered by the OS:
 - example : pipes, send-receive via sockets, ...
- *They have higher communication overhead than using shared-variables!*

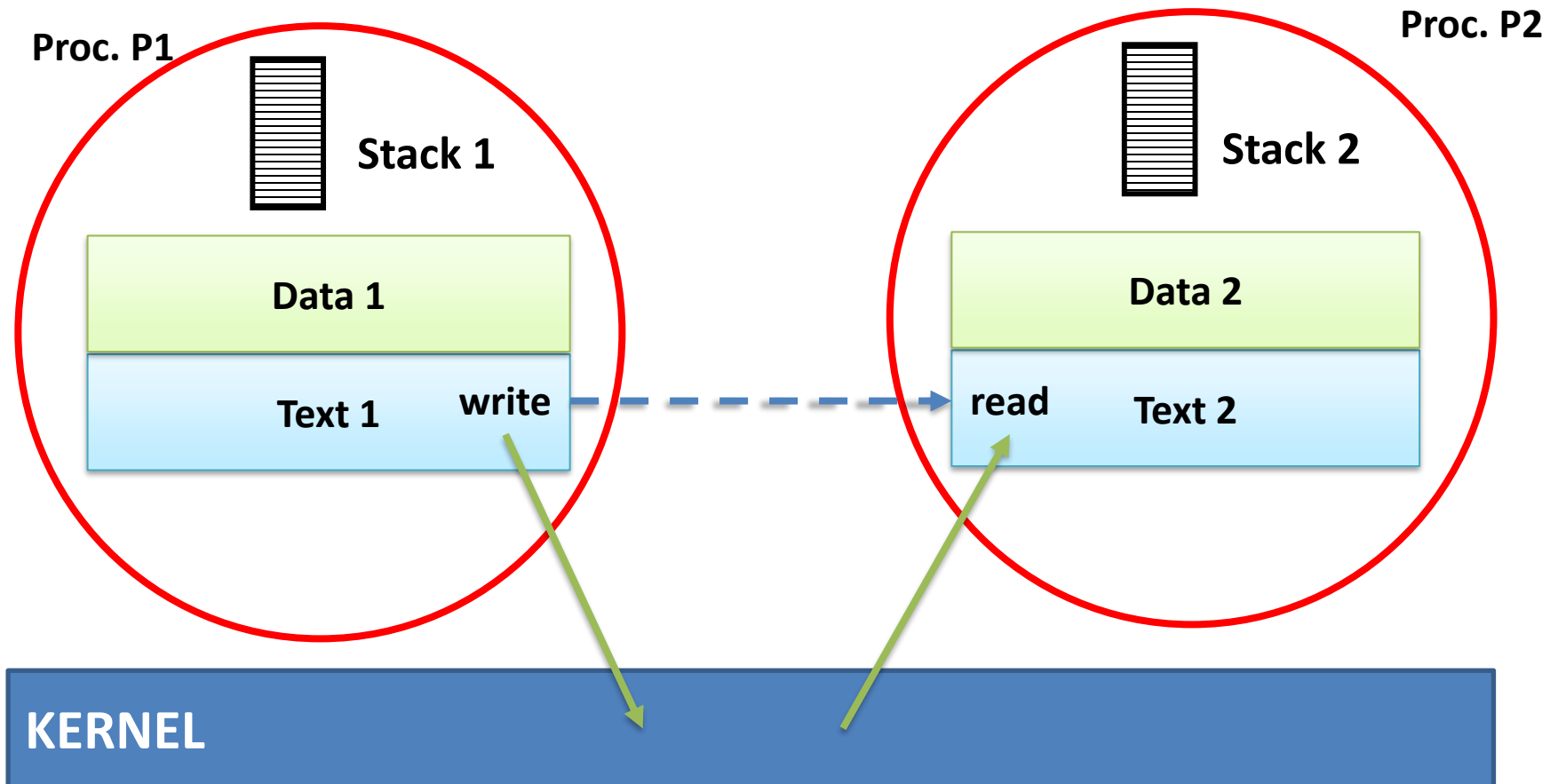
Inter-process communications – IPC

- Two processes do not share memory!



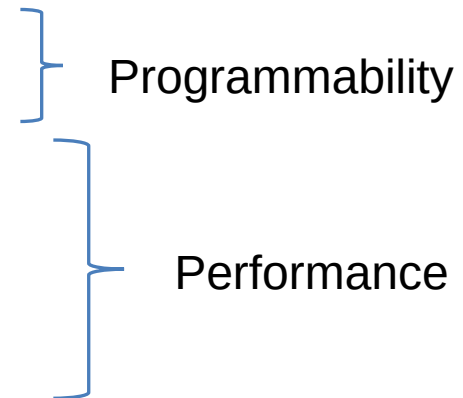
Inter-process communications

- Several mechanisms offered by the kernel
 - eg: write/read on pipes (named or unnamed pipes)
 - Performance issue: communications should pass through the kernel



Thread abstraction

- A thread is a single execution sequence that represents a ***separately schedulable task***
 - Single execution sequence: familiar programming model
 - Separately schedulable: OS can run or suspend a thread at any time
- Why using threads?
 - Program structure
 - Responsiveness
 - Exploiting multi-cores
 - Interacting with slow I/O devices

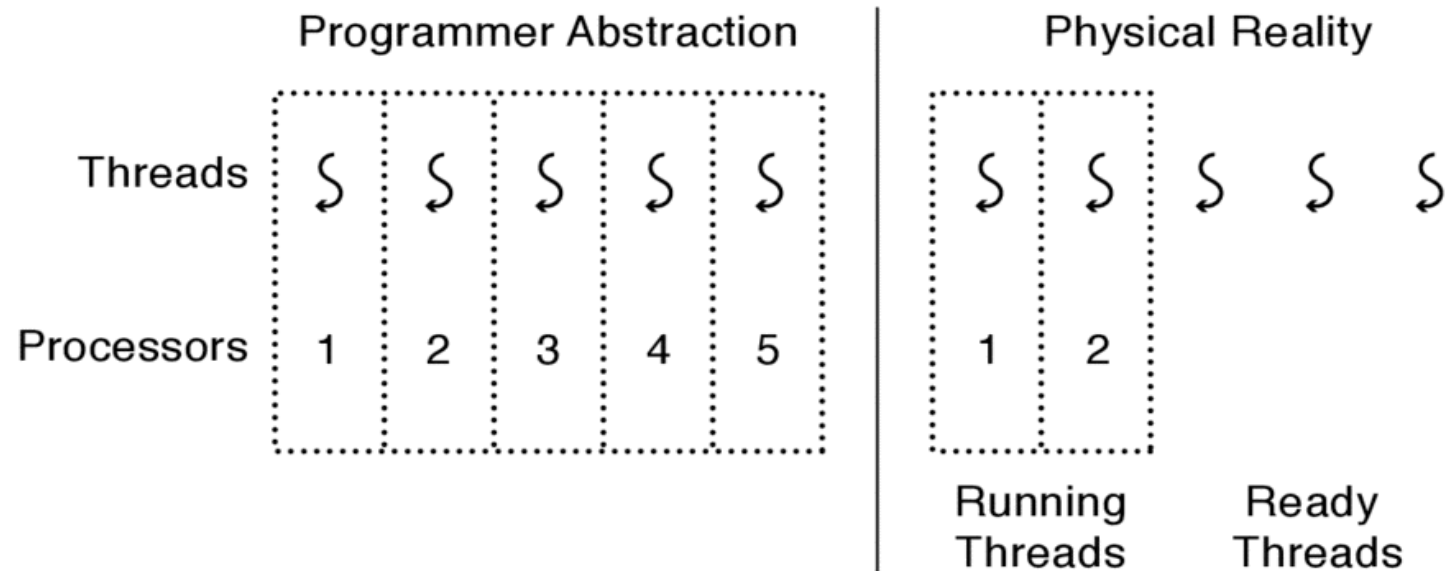


Thread abstraction

- Protection is an orthogonal concept
 - Can have one or many threads per protection domain (process)
 - Single threaded user program: one thread, one protection domain
 - Multi-threaded user program: multiple threads, sharing same data structures, isolated from other user processes
 - Multi-threaded kernel: multiple threads, sharing kernel data structures, capable of using privileged instructions

Thread Abstraction

- Unlimited number of processors
 - one processor for each thread
- Threads execute with variable speed
 - Programs must be designed to work with any schedule



Programmer vs. Processor View

Programmer's
View

.
.
.
x = x + 1;
y = y + x;
z = x + 5y;
.
.
.

Possible
Execution
#1

.
.
.
x = x + 1;
y = y + x;
z = x + 5y;
.
.
.

Possible
Execution
#2

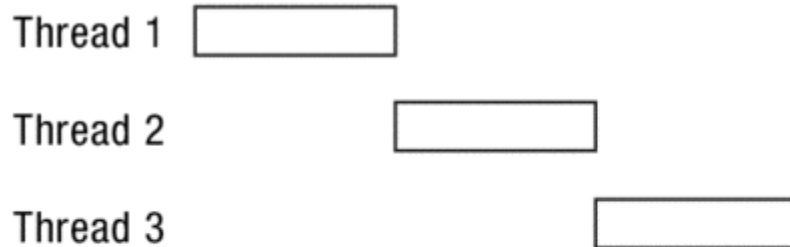
.
.
.
x = x + 1;
.....
Thread is suspended.
Other thread(s) run.
Thread is resumed.
.....
y = y + x;
z = x + 5y;

Possible
Execution
#3

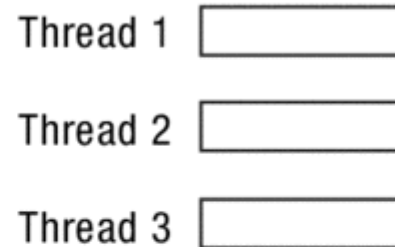
.
.
.
x = x + 1;
y = y + x;
.....
Thread is suspended.
Other thread(s) run.
Thread is resumed.
.....
z = x + 5y;

Possible Executions

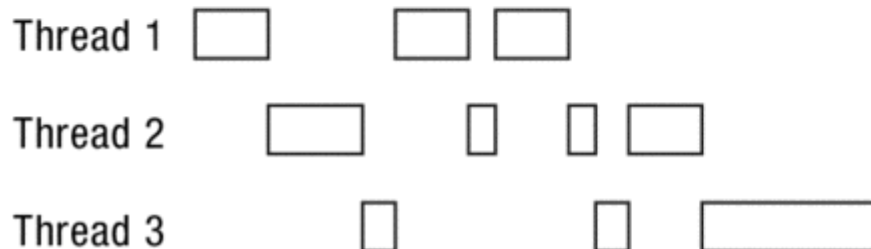
One Execution



Another Execution



Another Execution



Main: Fork 10 threads call join on them, then exit

- What other kinds of interleaving are possible?
- What is the maximum # of threads running at same time?
- Minimum?

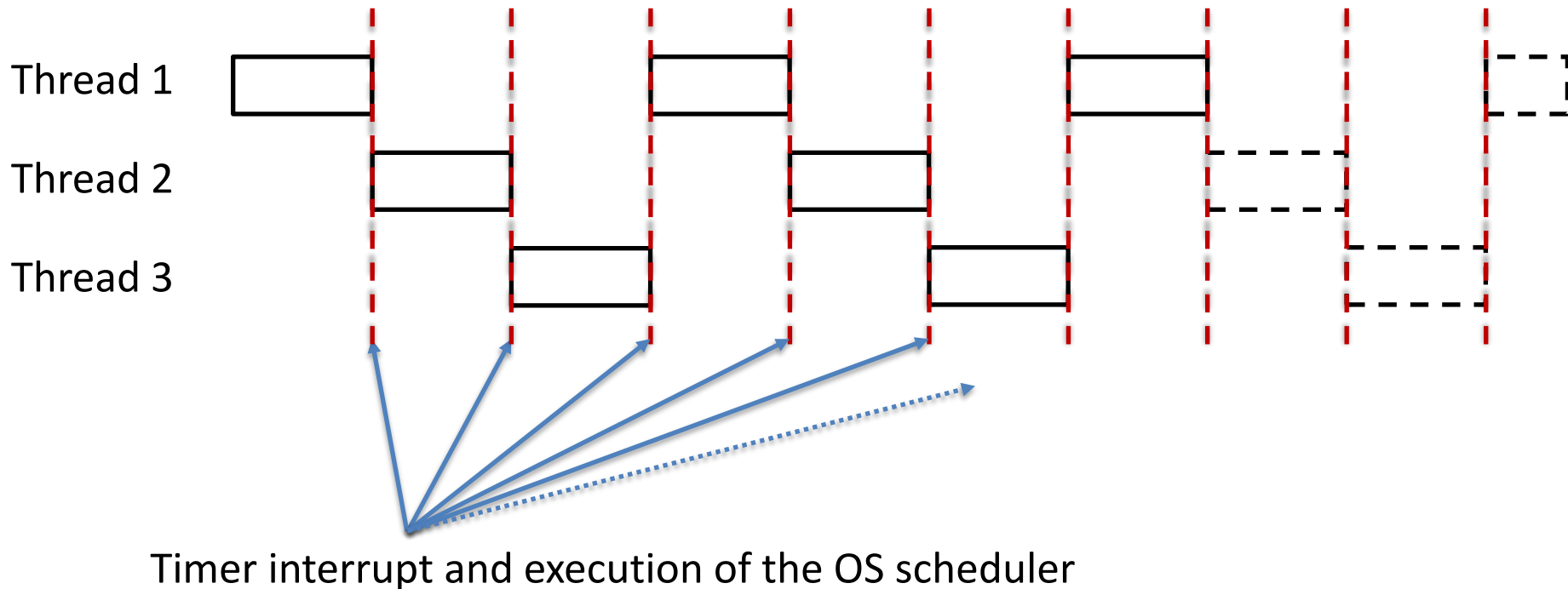
```
bash-3.2$ ./threadHello
Hello from thread 0
Hello from thread 1
Thread 0 returned 100
Hello from thread 3
Hello from thread 4
Thread 1 returned 101
Hello from thread 5
Hello from thread 2
Hello from thread 6
Hello from thread 8
Hello from thread 7
Hello from thread 9
Thread 2 returned 102
Thread 3 returned 103
Thread 4 returned 104
Thread 5 returned 105
Thread 6 returned 106
Thread 7 returned 107
Thread 8 returned 108
Thread 9 returned 109
Main thread done.
```

Implementing threads

- Thread Control Block (TCB) – data structure that stores information about the thread
- A set of operations on threads
- A scheduler
 - A function of the OS that assigns the processor(s) to the threads

Scheduler in time-sharing systems

** One processor, no interrupts from external devices*

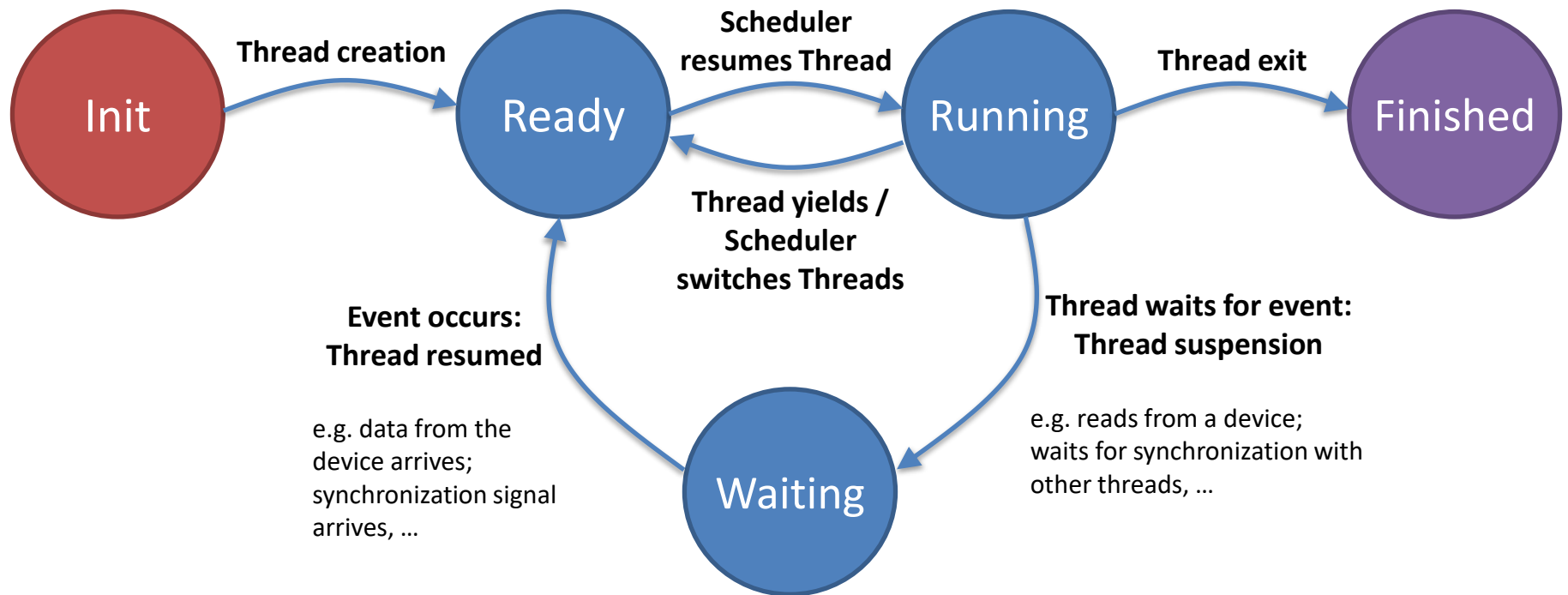


- *Cooperative vs preemptive* multi-threading

Simple Thread API

- `thread_create(thread, func, args)`
 - Creates a new thread, storing information about it in `thread`, to run `func(args)`
- `thread_yield()`
 - The calling thread voluntarily gives up the processor to let some other thread(s) run
- `thread_join(thread)`
 - Waits for `thread` to finish if it has not already done so, then returns the exit status of the thread. It can be called only once for each thread
- `thread_exit(exit_status)`
 - Finishes the current thread. Stores the `exit_status` in the thread's data structure. If another thread is waiting in the `thread_join`, resumes it.

Thread Lifecycle



Location of thread's per thread state

State of thread	Location of TCB	Location of registers
INIT	Being created	TCB
READY	Ready List	TCB
RUNNING	Running List	Processor
WAITING	Synchronization Variable's Waiting List	TCB
FINISHED	Finished List, then Deleted	TCB

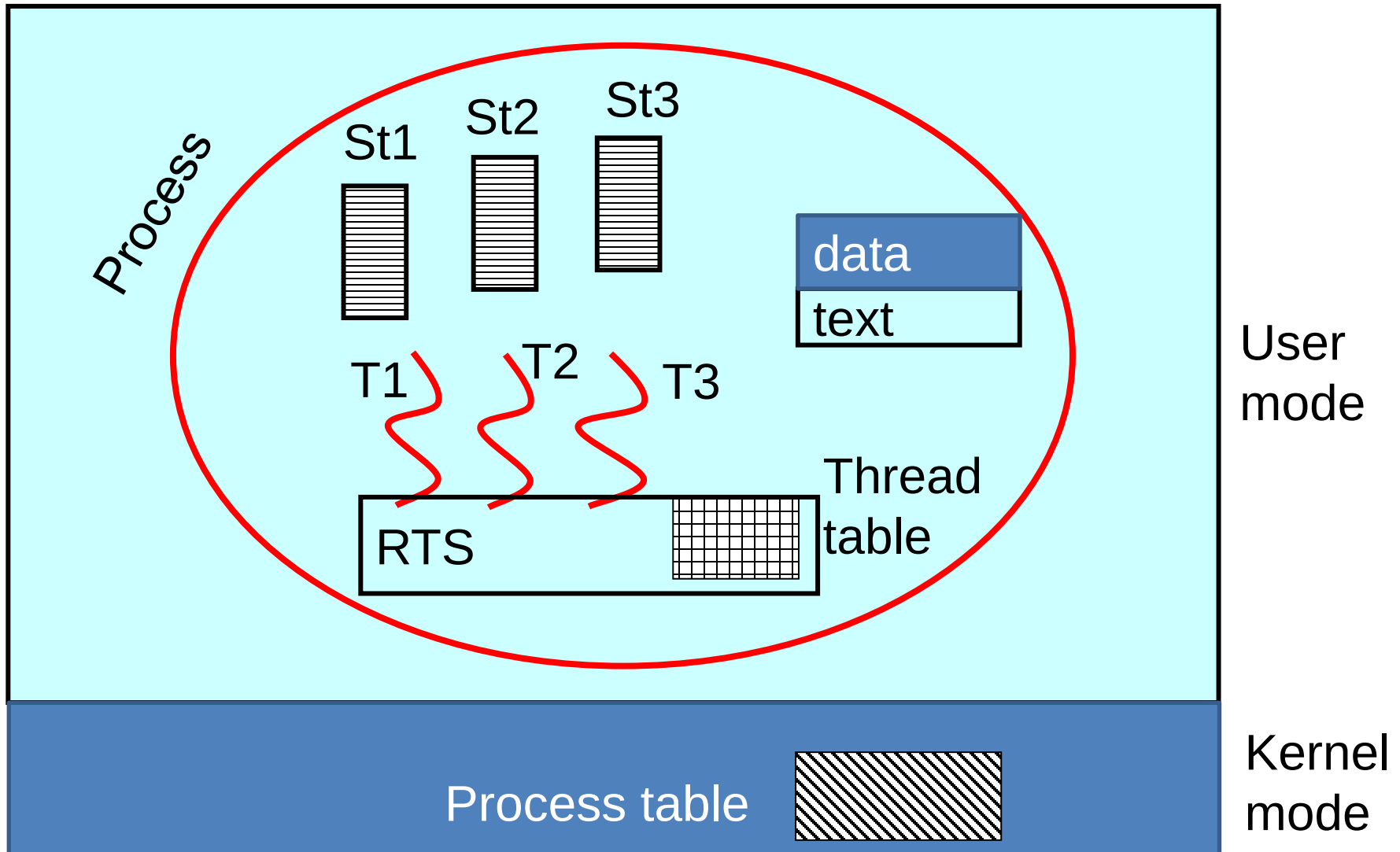
Roadmap

- Threads can be implemented in any of several ways
 - Multiple single-threaded processes (early UNIX)
 - Multiple user-level threads, inside a UNIX process (early Java)
 - Mixture of single and multi-threaded processes and kernel threads (Linux, MacOS, Windows)
 - To the kernel, a kernel thread and a single threaded user process look quite similar
 - Scheduler activations (Windows, see later)

User-level threads

- Threads implemented by means of a user-level library
- The O.S. is not aware of user level threads
- Thread table within each process (user space)
- Scheduling of the threads implemented by the Run Time Support (RTS) of the process
 - Threads can use *thread_yield()* to release the processor
 - *preemptive scheduler with Scheduler Activations in Windows
- *An invocation to a blocking system call blocks all the threads of the process*

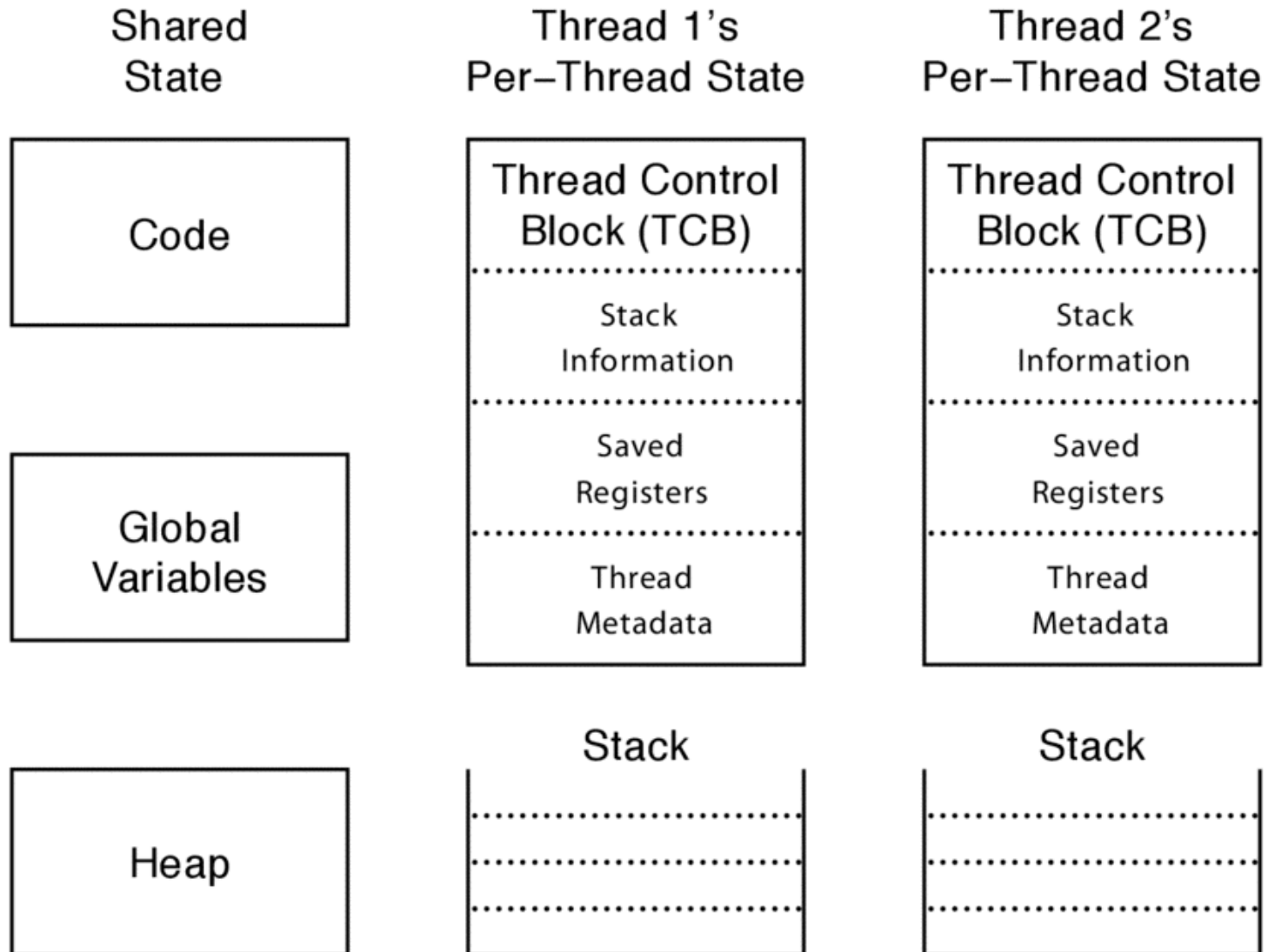
User-level threads



Implementing user-level threads

- `thread_create(thread, func, args)`
 - Allocate thread control block
 - Allocate stack
 - Build stack frame for base of stack (stub)
 - Put `func`, `args` on stack
 - Put thread on ready list
 - Will run sometime later (maybe right away!)
- `stub(func, args)`
 - Call `(*func)(args)`
 - Call `thread_exit()`

Shared vs. Per-Thread State



Thread Stack

- What if a thread puts too many procedures on its stack?
 - What should happen?
 - What happens in Java?
 - What happens in Linux?

User-level threads

Pros:

- Creation, termination and context switch very efficient
 - Do not need system call invocations, just calls to the thread library
 - In case of context switch the addressing space remains the same
- Can be implemented on any O.S. that does not support multithreading
 - e.g. early versions of UNIX

User-level threads

Cons (*):

- Blocking system calls block all the user-level threads of a process
- Do not take advantage of multiprocessors architectures
 - All threads of a process are scheduled on the same processor

* this does not apply to scheduler activations (see later)

Kernel-level threads

- Threads implemented in the kernel
 - Thread table in the kernel
 - Creation, termination and context switch activated by system calls
 - Different thread of the same process can run in parallel on different processors

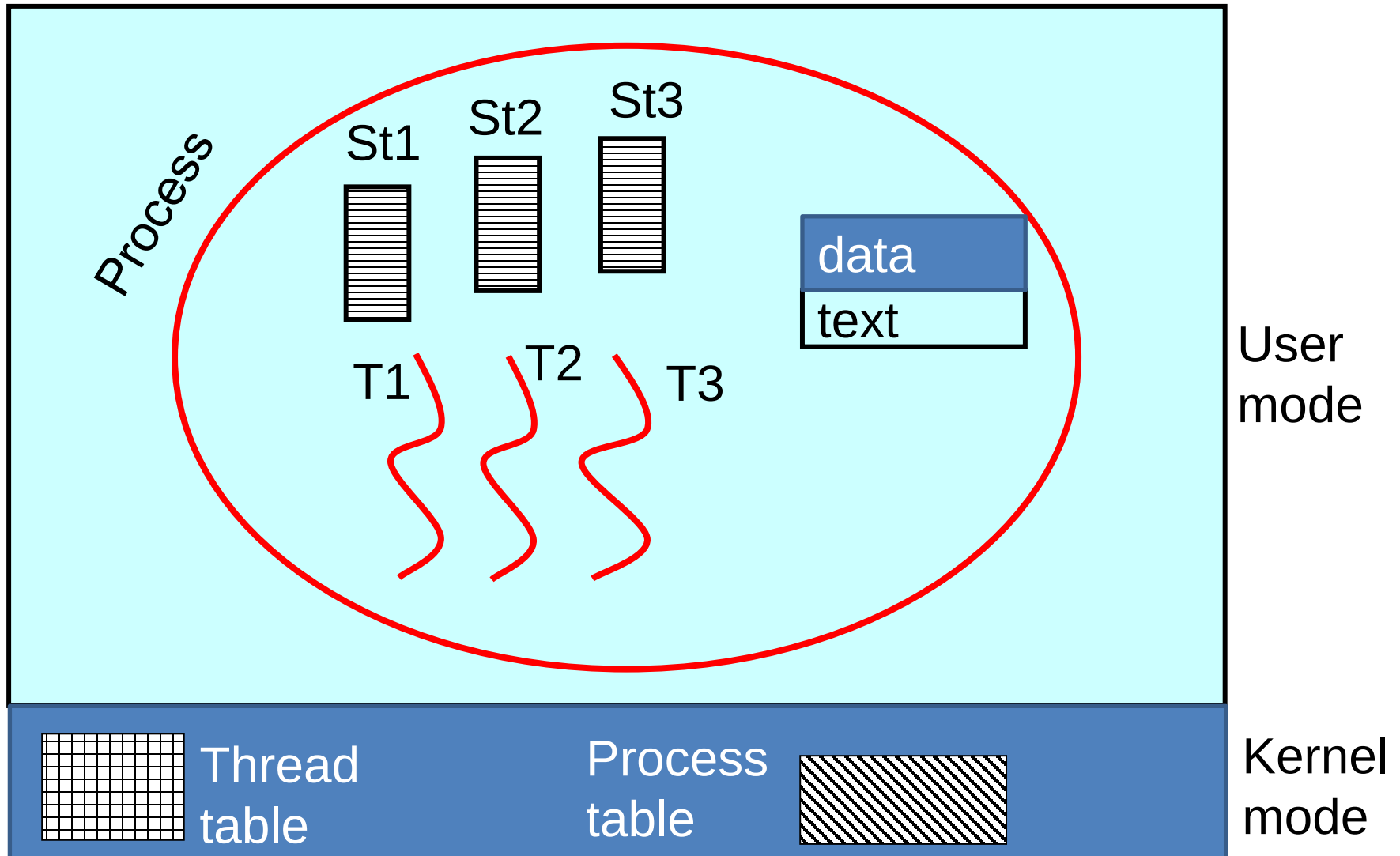
Processes and Threads representation

- Process Control Block (PCB)
 - Data structure associated to each process
- Process Table
 - Contains all PCBs
 - In the kernel, one for the entire system
- Thread Control Block (TCB)
 - One for each thread
- Thread Table
 - One for each process (user-level threads)
 - In the kernel, one for the entire system (kernel-level threads)

TCB & PCB

- PCB:
 - Process name (PID)
 - Assigned memory
 - Other resources
 - Devices, open files, ...
 - Handlers to the process' threads
 - ...
- TCB:
 - Thread ID
 - State
 - Context of the thread
 - Scheduling parameters
 - Reference to the stack(s)
 - ...

Kernel-level threads



Kernel-level threads

- Operations on threads and interactions among threads by means of System Calls
 - More overhead w.r.t. user-level threads
- Thread scheduling implemented by the O.S.
- Threads can invoke blocking system calls
 - Only the invoker gets blocked

Threads in a Process

- Threads are useful in user-level programs:
 - Parallelism, hide I/O latency, interactivity/responsiveness
- Option A (early Java): user-level library, within a single-threaded process
 - Library implements thread context switch
 - Kernel time slices between processes, e.g., on system call I/O
 - Main Issues: blocking system calls, non-preemptive scheduling
- Option B (Linux, MacOS, Windows): use kernel threads
 - System calls for thread fork, join, exit (and lock, unlock,...)
 - Kernel implements context switch (preemptive scheduling)
 - Simple, but a lot of transitions between user and kernel mode

Threads in a Process

- Option C (Windows): *Scheduler Activations*
 - Kernel allocates processors to user-level library
 - Thread library implements context switch
 - System call I/O that blocks triggers upcall
- Option D: Asynchronous I/O
- Option E: Threads + Asynchronous I/O

Thread Switch

- Two causes:
 - Voluntary
 - Due to an interrupt/exception
- Almost the same management for the different cases:
 - Kernel/user threads
 - Multithread/singlethread processes

Implementing (voluntary) thread context switch

- User-level threads in a user-level process:
 - Save registers on old TCB (of the running thread)
 - It may temporarily save registers on the stack
 - Switch to new stack & to new thread
 - Restore registers from new thread's TCB
 - return
- Kernel threads
 - Exactly the same, 'return' substituted by proper instruction (e.g., IRET or MOVS/SUBS)

Two threads call thread_yield()

Thread 1's instructions

call thread_yield
save state to stack
save state to TCB
choose another thread
load other thread state

return thread_yield
call thread_yield
save state to stack
save state to TCB
choose another thread
load other thread state

return thread_yield

...

Thread 2's instructions

call thread_yield
save state to stack
save state to TCB
choose another thread
load other thread state

return thread_yield
call thread_yield
save state to stack
save state to TCB
choose another thread
load other thread state

...

Processor's instructions

call thread_yield
save state to stack
save state to TCB
choose another thread
load other thread state

call thread_yield
save state to stack
save state to TCB
choose another thread
load other thread state

return thread_yield
call thread_yield
save state to stack
save state to TCB
choose another thread
load other thread state

return thread_yield
call thread_yield
save state to stack
save state to TCB
choose another thread
load other thread state
return thread_yield

...

Thread switch on an interrupt

- Thread switch can occur due to timer or I/O interrupt
 - Tells OS some other thread should run
- Simple version
 - Interrupt handler first save registers in kernel stack
 - End of interrupt handler calls **switch_threads()**
 - When resumed, return from handler resumes kernel thread or user process/thread
- Faster version...
 - Interrupt handler saves registers directly in TCB
 - Interrupt handler returns to saved state in TCB
 - Resumes a kernel thread or a user process/thread

switch_threads()

- Save registers (context) of the old thread from kernel stack in the TCB
- Move old thread's TCB to Ready List or to a Waiting List
- Select a new thread from the Ready List
 - scheduling problem – discussed later
- Restores new thread's registers from TCB to processor
- Put new thread's TCB in the Running List
- return control to the new thread (IRET...)

Thread switch - overhead

- Due to registers save and restore
- Due to TCB queues management
- Memory cache invalidation
 - Refer to the computer architecture classes...
- Induced operations on the memory manager
 - Address exceptions
 - Page faults
 - MMU invalidation
 - Will be discussed later on

Context switch - example

Let us consider again a simplified processor with special registers PC and PS, the user-level stack pointer SP and just two general registers R1, R2. The interrupt vector is in memory. The system uses a single kernel stack (shared for all threads). IRET to return. When it receives an interrupt, the processor:

- Sets kernel mode;
- Disables interrupts;
- Saves PC & PS & SP on the kernel stack
- Loads the new PC & PS from the interrupt vector
- Consequently it jumps to the interrupt handler in the kernel

Hardware

The IRET instruction:

- Enables interrupts;
- Sets user mode;
- Restores PC, PS & SP from the kernel stack; (consequently jumps back to the address at which the RUNNING thread had been interrupted in the past)

The interrupt handler:

- First saves the general registers on the kernel stack
- At the end, it restores the general registers and then executes IRET

Software

Context switch – example 1

Hyp. A): thread T1 invokes a system call. At the end it remains in RUNNING state

1) Initial situation during the execution of SVC instruction (**USER MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	
0FFE	
0FFD	
0FFC	
0FFB	
0FFA	

Registers	
PC	1880
PS	16F2
SP	2880
R1	4500
R2	CD31

address	5000
PS	AA45
Interrupt vector	

base kernel SP	0FFF
----------------	------

Context switch - example

1) Initial situation during the execution of SVC instruction (**USER MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	
0FFE	
0FFD	
0FFC	
0FFB	
0FFA	

Registers	
PC	1880
PS	16F2
SP	2880
R1	4500
R2	CD31

address	5000
PS	AA45
Interrupt vector	

Base kernel SP	0FFF
----------------	------

2) After interrupt (**KERNEL MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	1880
0FFE	16F2
0FFD	2880
0FFC	
0FFB	
0FFA	

Registers	
PC	5000
PS	AA45
SP	0FFC
R1	4500
R2	CD31

address	5000
PS	AA45
Interrupt vector	

Base kernel SP	0FFF
----------------	------

Context switch - example

2) After interrupt (**KERNEL MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	1880
0FFE	16F2
0FFD	2880
0FFC	
0FFB	
0FFA	

Registers	
PC	5000
PS	AA45
SP	0FFC
R1	4500
R2	CD31

3) After temporary storage of registers (**KERNEL MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	1880
0FFE	16F2
0FFD	2880
0FFC	4500
0FFB	CD31
0FFA	

Registers	
PC	5000 + ?
PS	AA45
SP	0FFA
R1	??
R2	??

Context switch - example

4) At the end of the primitive (**KERNEL MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	1880
0FFE	16F2
0FFD	2880
0FFC	4500
0FFB	CD31
0FFA	

Registers	
PC	5000 + ?
PS	AA45
SP	OFFA
R1	??
R2	??

5) During extraction of IRET at address 5100 (**KERNEL MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	1880
0FFE	16F2
0FFD	2880
0FFC	
0FFB	
0FFA	

Registers	
PC	5100
PS	AA45
SP	OFFC
R1	4500
R2	CD31

Context switch - example

5) During execution of IRET at address 5100 (**KERNEL MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	1880
0FFE	16F2
0FFD	2880
0FFC	
0FFB	
0FFA	

Registers	
PC	5100
PS	AA45
SP	OFFC
R1	4500
R2	CD31

6) At the end of IRET (**USER MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	
0FFE	
0FFD	
0FFC	
0FFB	
0FFA	

Registers	
PC	1880
PS	16F2
SP	2880
R1	4500
R2	CD31

Context switch – example 2

Hyp. B): thread T1 invokes a system call that blocks T1 and switches T2 in RUNNING state

1) Initial situation during the execution of SVC instruction (**USER MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	
0FFE	
0FFD	
0FFC	
0FFB	
0FFA	

Registers	
PC	1880
PS	16F2
SP	2880
R1	4500
R2	CD31

address	5000
PS	AA45
Interrupt vector	

base kernel SP	0FFF
----------------	------

Context switch - example

1) Initial situation during the execution of SVC instruction (**USER MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	
0FFE	
0FFD	
0FFC	
0FFB	
0FFA	

Registers	
PC	1880
PS	16F2
SP	2880
R1	4500
R2	CD31

address	5000
PS	AA45
Interrupt vector	

Base kernel SP	0FFF
----------------	------

2) After interrupt (**KERNEL MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	1880
0FFE	16F2
0FFD	2880
0FFC	
0FFB	
0FFA	

Registers	
PC	5000
PS	AA45
SP	0FFC
R1	4500
R2	CD31

address	5000
PS	AA45
Interrupt vector	

Base kernel SP	0FFF
----------------	------

Context switch - example

2) After interrupt (**KERNEL MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	1880
0FFE	16F2
0FFD	2880
0FFC	
0FFB	
0FFA	

Registers	
PC	5000
PS	AA45
SP	0FFC
R1	4500
R2	CD31

3) After temporary storage of registers (**KERNEL MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	1880
0FFE	16F2
0FFD	2880
0FFC	4500
0FFB	CD31
0FFA	

Registers	
PC	5000 + ?
PS	AA45
SP	0FFA
R1	??
R2	??

Context switch - example

3) After temporary storage of registers (**KERNEL MODE**)

TCB T1	
State	Running
PC	????
PS	16F2
SP	????
R1	????
R2	????

TCB T2	
State	Ready
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	1880
0FFE	16F2
0FFD	2880
0FFC	4500
0FFB	CD31
0FFA	

Registers	
PC	5000 + ?
PS	AA45
SP	OFFA
R1	??
R2	??

4) After storage of registers of T1 and restore of registers of T2 (**KERNEL MODE**)

TCB T1	
State	Waiting
PC	1880
PS	16F2
SP	2880
R1	4500
R2	CD31

TCB T2	
State	Running
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	A12C
0FFE	16F2
0FFD	A275
0FFC	25CC
0FFB	F012
0FFA	

Registers	
PC	5000+?
PS	AA45
SP	OFFA
R1	??
R2	??

Context switch - example

4) After storage of registers of T1 and restore of registers of T2 (**KERNEL MODE**)

TCB T1	
State	Waiting
PC	1880
PS	16F2
SP	2880
R1	4500
R2	CD31

TCB T2	
State	Running
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	A12C
0FFE	16F2
0FFD	A275
0FFC	25CC
0FFB	F012
0FFA	

Registers	
PC	5000+?
PS	AA45
SP	OFFA
R1	??
R2	??

5) During execution of IRET at address 5100 (**KERNEL MODE**)

TCB T1	
State	Waiting
PC	1880
PS	16F2
SP	2880
R1	4500
R2	CD31

TCB T2	
State	Running
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	A12C
0FFE	16F2
0FFD	A275
0FFC	
0FFB	
0FFA	

Registers	
PC	5100
PS	AA45
SP	OFFC
R1	25CC
R2	F012

Context switch - example

5) During execution of IRET at address 5100 (**KERNEL MODE**)

TCB T1	
State	Waiting
PC	1880
PS	16F2
SP	2880
R1	4500
R2	CD31

TCB T2	
State	Running
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	A12C
0FFE	16F2
0FFD	A275
0FFC	
0FFB	
0FFA	

Registers	
PC	5100
PS	AA45
SP	OFFC
R1	25CC
R2	F012

6) At the end of IRET (**USER MODE**)

TCB T1	
State	Waiting
PC	1880
PS	16F2
SP	2880
R1	4500
R2	CD31

TCB T2	
State	Running
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012

Kernel stack	
0FFF	
0FFE	
0FFD	
0FFC	
0FFB	
0FFA	

Registers	
PC	A12C
PS	16F2
SP	A275
R1	25CC
R2	F012